

# 逻辑回归与数据处理核心知识点

## 逻辑回归与数据处理核心知识点文档

### 引言

本文档整理了逻辑回归模型应用过程中的核心知识点，涵盖模型原理、特征工程、数据处理、可视化及训练评估等内容，旨在为逻辑回归实践提供清晰指引。

## 一、逻辑回归模型原理与参数解析

### 1. 模型数学基础

逻辑回归是基于**sigmoid函数**的二分类模型，通过将线性组合结果映射到[0,1]区间表示分类概率：

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n)}}$$

- 当  $(h_{\theta}(x) \geq 0.5)$  时，预测为类别1（如“通过”）；
- 当  $(h_{\theta}(x) < 0.5)$  时，预测为类别0（如“未通过”）。

### 2. 核心参数含义

- coef\_**：特征系数数组，形状为  $(n\_classes, n\_features)$ （二分类时为  $(1, n\_features)$ ）。
  - 例：**coef\_[0][0]** 对应第一个特征（如 Exam1）的系数  $(\theta_1)$ ，**coef\_[0][1]** 对应第二个特征（如 Exam2）的系数  $(\theta_2)$ ；
  - 系数正负表示特征对分类的影响方向（正系数：特征值越大，越可能属于类别1）。

- `intercept_`：截距项（偏置项），对应公式中的  $(\theta_0)$ ，二分类时为标量。

### 3. 预测方法

- `predict(X)`：输出类别标签（0/1），基于 `sigmoid` 概率的0.5阈值判断；
- `predict_proba(X)`：输出样本属于各类别的概率（二分类时为两列，分别对应0和1的概率），支持自定义阈值调整分类策略。

## 二、特征工程：从线性到非线性拟合

### 1. 线性模型的局限性

逻辑回归是线性模型，默认决策边界为线性函数（如二维特征时为直线  $(\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0)$ ），无法拟合非线性分布数据（如曲线分隔的样本）。

### 2. 非线性特征的构造

通过创建高次项或交叉项，可将线性模型转化为“非线性组合的线性拟合”，从而拟合非线性边界：

- 常见非线性特征：
  - 平方项： $(x_1^2 = x_1 \times x_1)$ 、 $(x_2^2 = x_2 \times x_2)$ ；
  - 交叉项： $(x_1 x_2 = x_1 \times x_2)$ 。
- 手动构造示例：

Code block

```
1  X1 = data['Exam1']  # 原始特征1
2  X2 = data['Exam2']  # 原始特征2
3  X1_2 = X1 * X1      # 平方项
4  X2_2 = X2 * X2      # 平方项
5  X1_X2 = X1 * X2     # 交叉项
6  # 合并为新特征矩阵
```

```
7 X_new = pd.DataFrame({'X1': X1, 'X2': X2, 'X1^2': X1_2, 'X2^2': X2_2,
    'X1X2': X1_X2})
```

### 3. 自动生成非线性特征 (PolynomialFeatures)

sklearn的 PolynomialFeatures 可自动生成指定次数的多项式特征，简化流程：

Code block

```
1 from sklearn.preprocessing import PolynomialFeatures
2 # degree=2: 生成最高二次项 (含原始特征、平方项、交叉项)
3 poly = PolynomialFeatures(degree=2, include_bias=False) # 不生成偏置项
4 X_new = poly.fit_transform(X_original) # X_original为原始特征矩阵 (n_samples,
    2)
```

## 三、数据处理工具：pandas核心操作

### 1. pd.DataFrame 详解

pd.DataFrame 是二维表格数据结构，由行索引、列索引和数据值组成，用于存储结构化数据。

- 创建方式：

- 从字典创建：键为列名，值为同长度列表/数组（需保证列长一致）：

Code block

```
1 data = {'Exam1': [34.62, 30.29], 'Exam2': [78.02, 43.90], 'Pass': [0, 0]}
2 df = pd.DataFrame(data)
```

- 从列表创建：需通过 columns 指定列名：

Code block

```
1 df = pd.DataFrame([[34.62, 78.02, 0]], columns=['Exam1', 'Exam2',  
          'Pass'])
```

- **注意事项：**输入数据需为二维结构（单样本用 `[[x1, x2]]`），否则因维度不匹配报错。

## 2. 数据筛选与 `mask` 的使用

- **`loc` 标签索引：**通过列名筛选数据（如 `data.loc[:, 'Exam1']` 取所有行的 `Exam1` 列）。
- **`mask` 的作用：**`mask` 是自定义布尔数组，用于标记符合条件的样本：

Code block

```
1 mask = data['Pass'] == 1 # 筛选“通过”样本 (Pass=1)  
2 passed_samples = data[mask] # 取mask为True的行  
3 failed_samples = data[~mask] # 取mask为False的行 (~表示取反)
```

## 3. 排序操作： `sort_values`

- **作用：**按指定列值排序（默认升序），确保可视化时x轴特征从左到右递增，避免决策边界曲线错乱。
- **示例：**

Code block

```
1 X1_sorted = data['Exam1'].sort_values() # 对Exam1列升序排序
```

# 四、可视化：matplotlib绘图与决策边界

## 1. 基础绘图流程

```

1 import matplotlib.pyplot as plt
2 # 创建画布
3 fig = plt.figure(figsize=(8, 6))
4 # 绘制散点图 (区分两类样本)
5 mask = data['Pass'] == 1
6 plt.scatter(data.loc[mask, 'Exam1'], data.loc[mask, 'Exam2'], c='blue',
7             label='passed')
8 plt.scatter(data.loc[~mask, 'Exam1'], data.loc[~mask, 'Exam2'], c='orange',
9             label='failed')
10 # 添加标签与标题
11 plt.xlabel('Exam1')
12 plt.ylabel('Exam2')
13 plt.title('Exam1 vs Exam2') # 注意拼写: title (非titile)
14 plt.legend() # 显示图例
15 plt.show()

```

## 2. 决策边界绘制

### (1) 线性边界 (原始特征)

由  $(\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0)$  推导:

$$x_2 = -(\theta_0 + \theta_1 x_1) / \theta_2$$

Code block

```

1 theta0 = LR.intercept_[0]
2 theta1, theta2 = LR.coef_[0][0], LR.coef_[0][1]
3 X1_sorted = data['Exam1'].sort_values()
4 X2_boundary = - (theta0 + theta1 * X1_sorted) / theta2
5 plt.plot(X1_sorted, X2_boundary, 'r-') # 绘制直线边界

```

### (2) 非线性边界 (含高次特征)

基于二次项特征的边界公式为二次曲线，需通过解方程计算边界点，最终绘制平滑曲线（如用户案例中的蓝色曲线）。

## 五、模型训练与评估

### 1. 训练流程

Code block

```
1  from sklearn.linear_model import LogisticRegression
2  from sklearn.metrics import accuracy_score
3
4  # 准备数据
5  X = data[['Exam1', 'Exam2']]
6  y = data['Pass']
7
8  # 创建模型（解决收敛问题：增加迭代次数）
9  LR = LogisticRegression(max_iter=1000)
10
11 # 训练与评估
12 LR.fit(X, y)
13 y_pred = LR.predict(X)
14 accuracy = accuracy_score(y, y_pred) # 准确率：正确分类样本占比
```

### 2. 常见问题与解决

- **收敛警告（`ConvergenceWarning`）：**
  - 原因：优化器（如 `lbfgs`）迭代次数不足或特征尺度差异大；
  - 解决：增加 `max_iter`（如 `max_iter=1000`）或标准化特征（`StandardScaler`）。
- **特征名称不匹配警告：**
  - 原因：训练用带列名的DataFrame，预测输入为纯数组；
  - 解决：预测时用同列名的DataFrame输入（如 `pd.DataFrame([[70,65]], columns=['Exam1','Exam2'])`）。

## 六、常见错误与解决方案

| 错误类型                                                                       | 原因                                | 解决方法                               |
|----------------------------------------------------------------------------|-----------------------------------|------------------------------------|
| <code>AttributeError: 'matplotlib.pyplot' has no attribute 'titile'</code> | <code>plt.titile</code> 拼写错误      | 修正为 <code>plt.title()</code>       |
| <code>ValueError</code> (DataFrame 创建失败)                                   | 输入数据为一维 (如 <code>[70,65]</code> ) | 改为二维结构 (如 <code>[[70,65]]</code> ) |
| 收敛警告                                                                       | 迭代次数不足或特征尺度差异大                    | 增加 <code>max_iter</code> 或标准化特征    |

## 总结

本文档覆盖逻辑回归模型从原理到实践的核心知识点，包括模型参数解析、非线性特征构造、pandas 数据处理、matplotlib可视化及训练评估技巧，可作为逻辑回归实践的参考指南。

(注：文档部分内容可能由 AI 生成)